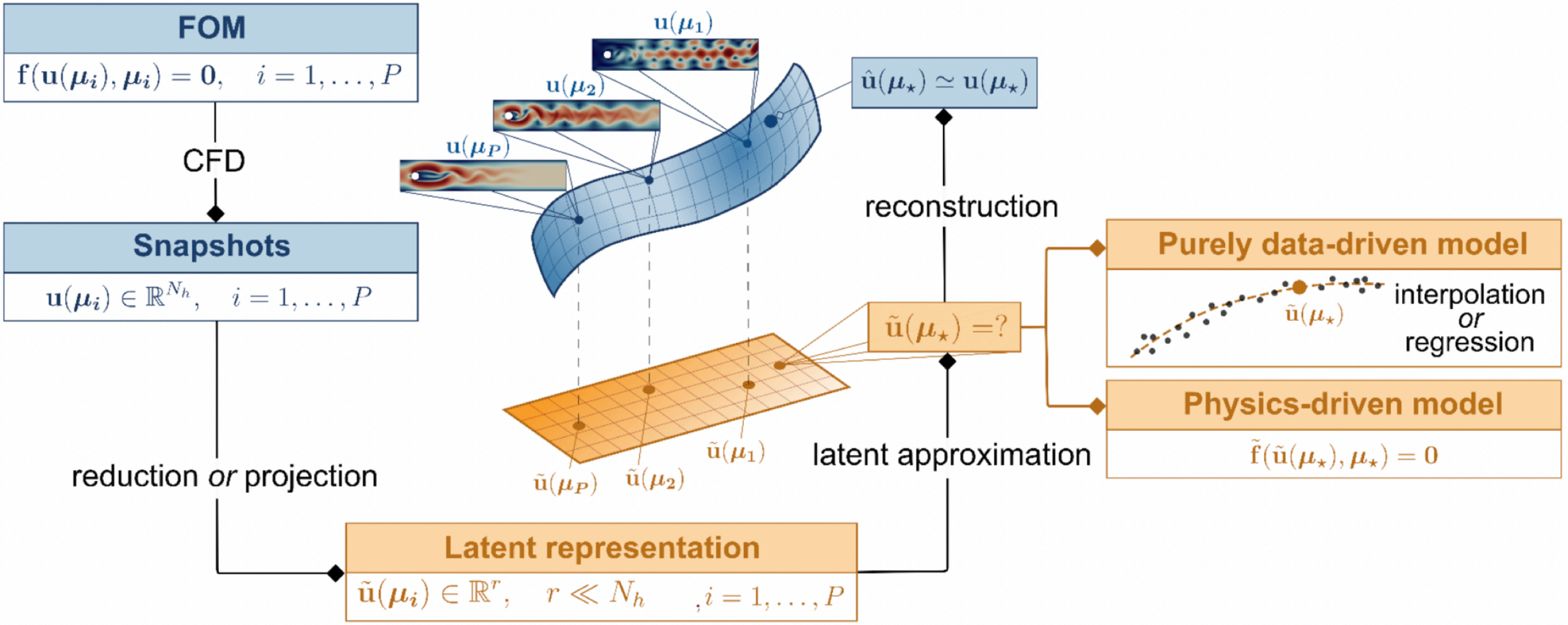
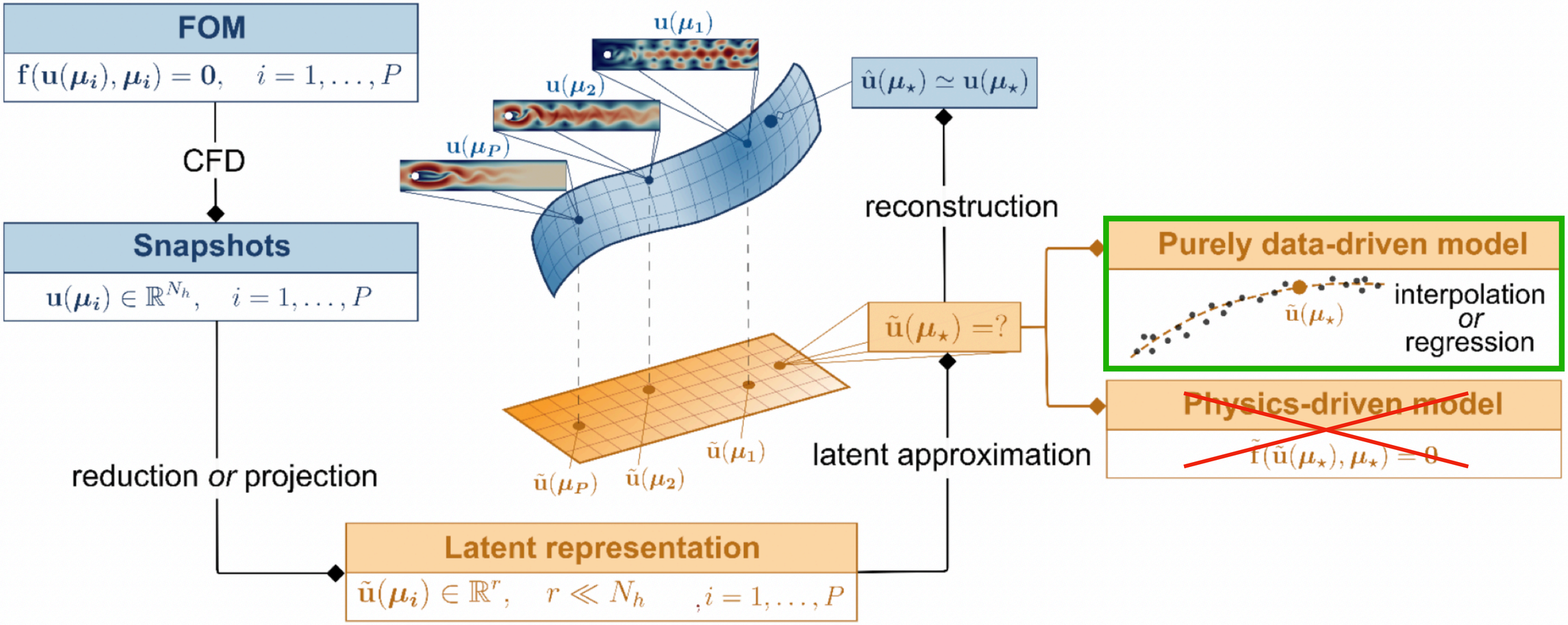
Reduced-Order Modelling

Non-intrusive ROMs



Reduced-Order Modelling

Non-intrusive ROMs



Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation
or
regression

The full pipeline: POD + regression

Offline: once, expensive

Sample parameters
$\mu_i, \quad i = 1, ..., n_{train}$

→

Run the FOM
$u(\mu_i) \in \mathbb{R}^{\mathcal{N}}$

→

Compress with POD
$u(\mu_i)_N = \mathbf{Z}^T u(\mu_i) \in \mathbb{R}^N$

→

Train the regressor
$\mu_i \to \tilde{u}(\mu_i)(RBF, GPR, ANN)$

Online: each new parameter, cheap

New parameter
μ^*

→

Evaluate the regressor
$\tilde{u}_N(\mu^*) = regressor(\mu^*)$

→

Reconstruct
$\hat{u}(\mu^*) = \mathbf{Z}\tilde{u}_N(\mu^*) \approx u(\mu^*)$

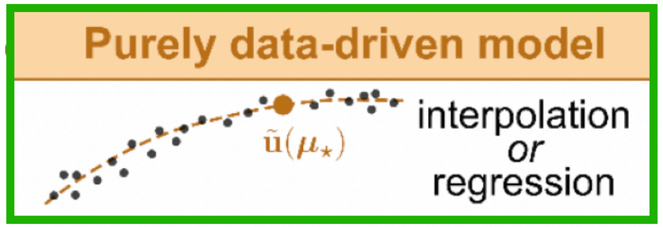
→

Output
Field or quantity of interest

Only snapshots and parameters are needed: no access to the FOM operators and no hyper-reduction. The online cost does not involve a FOM solve.

Reduced-Order Modelling

Non-intrusive ROMs

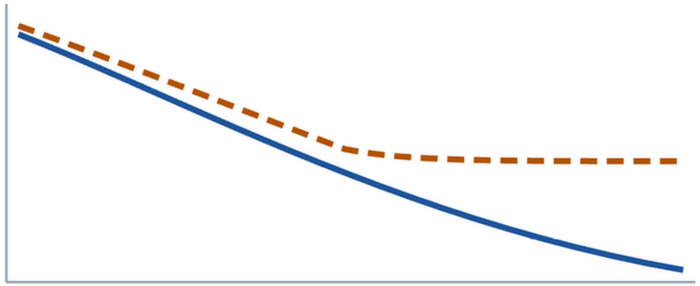


Error sources and validation

With orthonormal columns in V, the error splits into two orthogonal parts:

$$\|u - \hat{u}\|^2 = \|u - \mathbf{Z}\mathbf{Z}^T u\|^2 + \|\mathbf{Z}^T u - \tilde{u}\|$$

Projection error: set by the number of modes r and the decay of the singular values.
Regression error: set by the regressor, the number of snapshots P and their sampling.



Error vs number of modes r (log scale). Solid blue: projection error. Dashed orange: total error, which saturates at the regression error.

Sampling

Latin hypercube or Sobol points; greedy or GP-variance-driven refinement.

Validation

Split in μ into train, validation and test. Cross-validate ε, ℓ and the network size; never tune on the test set.

Metric

Relative L^2 error in the full space on unseen μ, reported next to the projection error.

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation
or
regression

Time-dependent problems

In an intrusive ROM the coefficients evolve through the projected equations. Without them, there are two options:

Time as a parameter

$$\tilde{u}_N(\mu^*) = regressor(\mu^*)$$

- **Pro:** any time queried directly, no error accumulation
- **Pro:** same pipeline as before, with input (μ, t)
- **Con:** cannot extrapolate beyond the training window
- **Con:** more training pairs: P × number of time steps

Learn the dynamics

$$\tilde{u}^{n+1} = F(\tilde{u}^n, \mu), \quad n = 0, 1, ...$$

- **Pro:** can march beyond the training window
- **Pro:** closest to the projected ODE of intrusive ROMs
- **Con:** errors accumulate over the roll-out
- **Con:** stability not guaranteed, sensitive to Δt

F can be an RBF, a GP or an ANN as before; other choices are DMD, SINDy, LSTMs and neural ODEs.

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation *or* regression

Learning the dynamics: a latent ODE

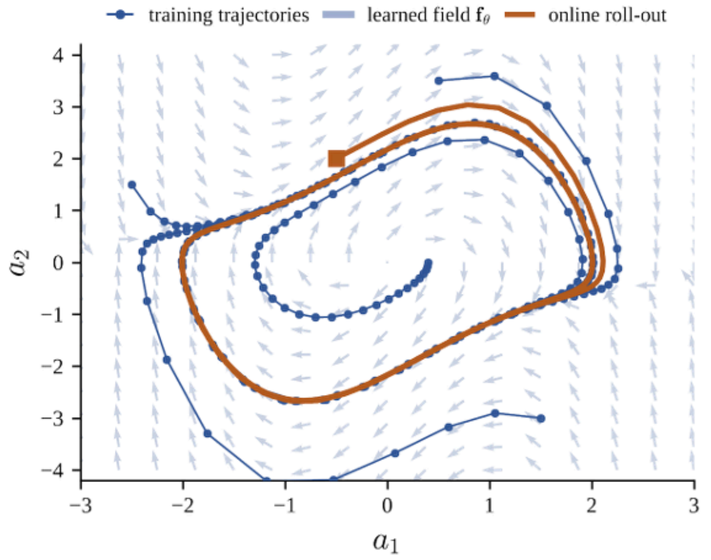
The intrusive ROM gives a closed ODE for the POD coefficients. Non-intrusively we keep this structure but learn its right-hand side from snapshots: given the state a (and μ), predict its time derivative.

Intrusive: operators known, projected ODE

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\,\dot{\mathbf{a}} = -\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{a})\mathbf{Z}\mathbf{a} - \tfrac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{a}$$

Non-intrusive: learn the vector field f(θ), data only

$$\dot{\tilde{\mathbf{a}}}(t) = \mathbf{f}_\theta(\tilde{\mathbf{a}}(t),\ \mu), \qquad \tilde{\mathbf{a}}(0) = \mathbf{Z}^T\mathbf{u}_0(\mu)$$



Illustrative 2-mode latent space (Van der Pol): the network is a vector field. trajectories are its integral curves.

Same regression tools as before *(RBF, GP, ANN)*: the map is now (a, μ) → ȧ instead of (μ, t) → a. Time integration is delegated to an ODE solver.

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation *or* regression

Training f(θ), option 1: derivative matching

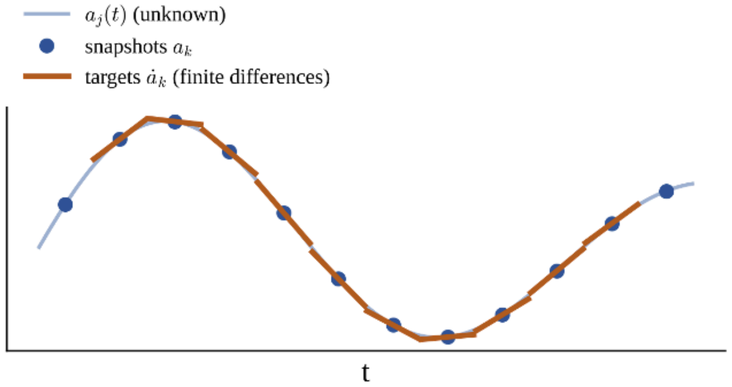
Treat f as one more regression problem: snapshots provide the states, finite differences in time provide the targets.

States (POD coefficients)

$$\mathbf{a}_{i,\,k} = \mathbf{Z}^T \mathbf{u}(t_k;\,\mu_i) \in \mathbb{R}^N$$

Targets (central differences)

$$\dot{\mathbf{a}}_{i,\,k} \approx \frac{\mathbf{a}_{i,\,k+1} - \mathbf{a}_{i,\,k-1}}{2\Delta t}$$



Regression loss, no ODE solve during training

$$\theta^{*} = \arg\min_{\theta} \frac{1}{n_{train} K} \sum_{i,\,k} \left\| \dot{\mathbf{a}}_{i,\,k} - \mathbf{f}_\theta(\mathbf{a}_{i,\,k},\,\mu_i) \right\|^2$$

Strengths

- Cheap: a plain (a, μ) → ȧ regression
- Any regressor works, no ODE solver in training

Limitations

- Finite differences amplify noise, need dense Δt
- Trained on slopes, deployed on roll-outs: errors accumulate

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation
or
regression

Training f(θ), option 1b: targets from the projected RHS

If the FOM right-hand side can be evaluated offline, the target ȧ is obtained exactly: no finite differences, no noise.

FOM semi-discrete system

$$\mathbf{M}\dot{\mathbf{u}} = \mathbf{r}(\mathbf{u};\mu) = -\mathbf{C}(\mathbf{u})\mathbf{u} - \tfrac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}$$

Target: the Galerkin-projected right-hand side

$$\dot{\mathbf{a}}_{i,\,k} = \left(\mathbf{Z}^T\mathbf{M}\mathbf{Z}\right)^{-1}\mathbf{Z}^T\mathbf{r}(\mathbf{Z}\mathbf{a}_{i,\,k};\mu_i)$$

Choices

- **Evaluate r at Z a:** the exact vector field of the Galerkin ROM; any state a can be sampled, not only trajectories
- **Evaluate r at u(t):** projection of the true derivative
- **Hybrid:** keep the linear part exact, learn only the nonlinear term

Strengths

- Exact, noise-free targets, independent of Δt
- More training pairs from RHS evaluations, no extra FOM runs

Limitations

- Needs the FOM operators offline: semi-intrusive training
- At best reproduces the Galerkin ROM, and its error

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation *or* regression

Training f(θ), option 2: neural ODE

Integrate f with an ODE solver and compare the predicted trajectory with the snapshots: no derivative estimation is needed.

Known state

a_k at time t_k

→ **ODE solver**

RK4 or adaptive, f_θ

→ **Prediction**

$\tilde{a}_{k+1}$ at time t_{k+1}

→ **Loss**

$\| a_{k+1} - \tilde{a}_{k+1} \|^2$

gradients: backprop through the solver, or adjoint ODE

$$\tilde{\mathbf{a}}_{k+1} = \tilde{\mathbf{a}}_k + \int_{t_k}^{t_{k+1}} \mathbf{f}_\theta(\tilde{\mathbf{a}}(s), \mu)\, ds$$

$$\mathcal{L}(\theta) = \frac{1}{n_{train} K} \sum_{i,\,k} \| \mathbf{a}_{i,\,k} - \tilde{\mathbf{a}}_{i,\,k} \|^2$$

Strengths

- Loss measures what we deploy: the roll-out
- Handles noisy, irregular snapshot times

Limitations

- Slower training: a solver call at every step
- Long windows: stiff, non-convex; use short windows

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation *or* regression

The full pipeline: POD + latent ODE

Offline: once, expensive

Sample parameters
$\mu_i, \quad i = 1, \ldots, n_{train}$

→

Run the FOM in time
$u(t_k; \mu_i) \in \mathbb{R}^{\mathcal{N}}$

→

Compress with POD
$\mathbf{a}_{i,k} = \mathbf{Z}^T \mathbf{u}(t_k; \mu_i)$

→

Train the latent ODE
f(θ): derivative matching or neural ODE

Online: each new parameter, cheap

Initial state
$\mathbf{a}_0^* = \mathbf{Z}^T \mathbf{u}_0(\mu^*)$

→

Integrate the ODE
$\tilde{\mathbf{a}}(t) = \mathrm{ODESolve}(\mathbf{f}_\theta, \mathbf{a}_0^*)$

→

Reconstruct
$\hat{u}(t; \mu^*) = \mathbf{Z}\tilde{\mathbf{a}}(t) \approx u(t; \mu^*)$

→

Output
Field in time, also beyond the training window

Only snapshots in time are needed: no FOM operators and no hyper-reduction. Online, the cost is an ODE solve in the small latent space, with free choice of the time step.

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model

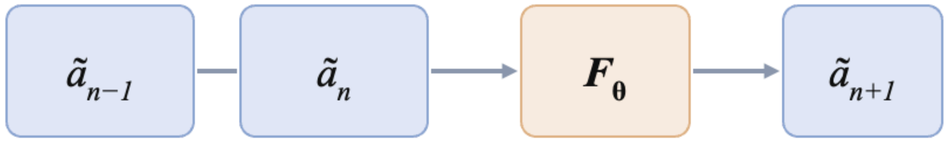


interpolation *or* regression

Autoregressive models on the POD coefficients

Skip the ODE: learn the discrete time-stepper directly, i.e. the map from the last w states to the next one.

$$\tilde{\mathbf{a}}_{n+1} = \mathbf{F}_\theta(\tilde{\mathbf{a}}_n, \tilde{\mathbf{a}}_{n-1}, \ldots, \tilde{\mathbf{a}}_{n-w+1};\ \mu)$$



window of w states — fed back at the next step

Windowed MLP (NARX)

$$\tilde{\mathbf{a}}_{n+1} = \tilde{\mathbf{a}}_n + \mathrm{MLP}_\theta(\tilde{\mathbf{a}}_n, \ldots, \tilde{\mathbf{a}}_{n-w+1}, \mu)$$

- Simplest: reuse the ANN of the previous slides
- Memory is the window w; residual form is more stable

Recurrent (LSTM, GRU)

$$\mathbf{h}_n = \mathrm{LSTM}_\theta(\mathbf{h}_{n-1}, [\tilde{\mathbf{a}}_n, \mu]), \quad \tilde{\mathbf{a}}_{n+1} = \mathbf{W}\mathbf{h}_n$$

- Hidden state h carries the history
- Sequential training; handles long memory

Causal Transformer

$$\tilde{\mathbf{a}}_{n+1} = \mathrm{Attn}_\theta(\tilde{\mathbf{a}}_{\leq n}, \mu) \quad \text{(causal mask)}$$

- Attention over all past steps, parallel training
- Data hungry; cost grows with the history length

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation
or
regression

Training and roll-out: the key difficulty

During training the network sees true states; online it must feed on its own predictions.

Teacher forcing (training): inputs are true states

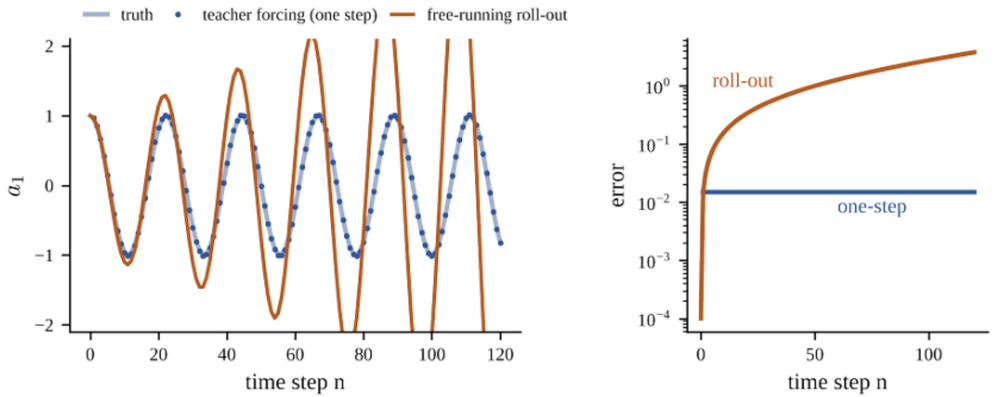
$$\mathcal{L}_{TF} = \sum_n \|\mathbf{a}_{n+1} - \mathbf{F}_\theta(\mathbf{a}_n, \ldots, \mathbf{a}_{n-w+1}; \mu)\|^2$$

Free-running (online): inputs are predictions

$$\tilde{\mathbf{a}}_{n+1} = \mathbf{F}_\theta(\tilde{\mathbf{a}}_n, \ldots, \tilde{\mathbf{a}}_{n-w+1}; \mu)$$

Remedies

- Multi-step loss over H unrolled steps
- Noise on the inputs; scheduled sampling
- Curriculum: grow H during training



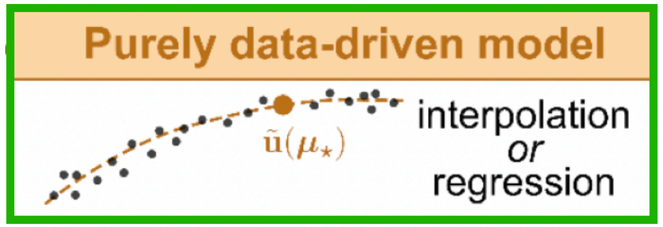
Illustrative: a small bias per step, invisible in one-step error, compounds over the roll-out (exposure bias).

$$\mathcal{L}_H = \sum_n \sum_{h=1}^{H} \|\mathbf{a}_{n+h} - \tilde{\mathbf{a}}_{n+h}\|^2$$

multi-step loss (needs backprop through time)

Reduced-Order Modelling

Non-intrusive ROMs



Latent ODE vs autoregressive

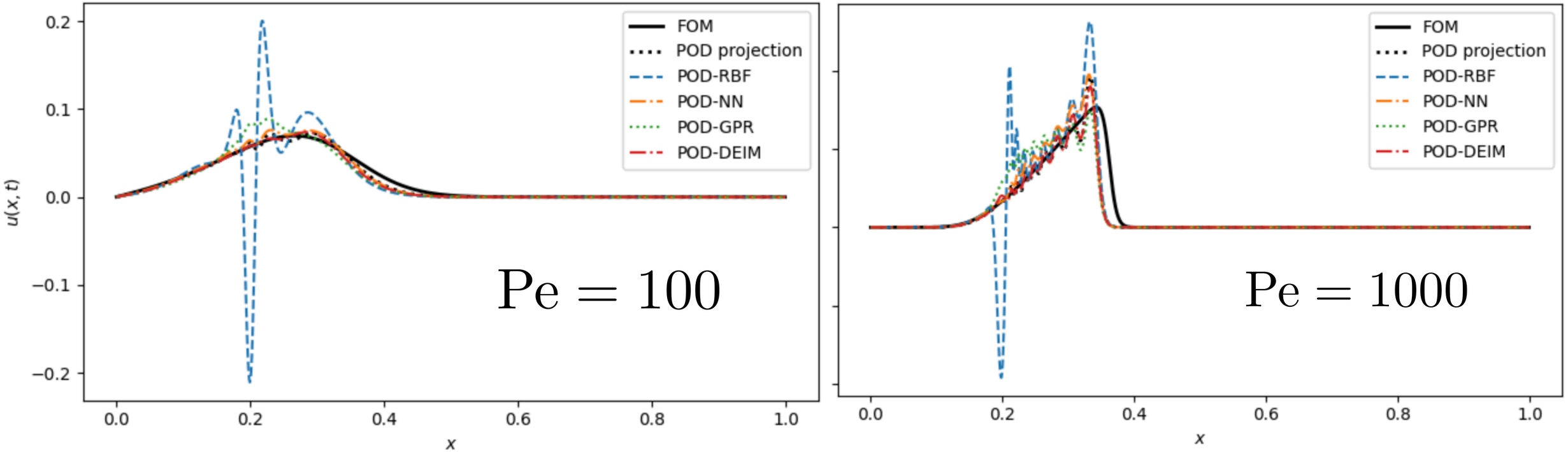
	Latent ODE	Autoregressive
Model	$\dot{a} = f(a, \mu)$	$a_{n+1} = F(a_n, \ldots, a_{n-w+1}, \mu)$
Time	Continuous; any Δt, adaptive solvers	Discrete; Δt fixed by the snapshots
Irregular snapshots	Natural	Needs resampling
Memory of the past	None in a; add state to get it	Window or hidden state
Training	Slopes (cheap) or solver (costly)	Teacher forcing (cheap, parallel)
Online cost	Several f evaluations per step	One forward pass per step
Main risk	Stiffness, noisy derivatives	Error accumulation

Rule of thumb: closed-looking dynamics and irregular or noisy sampling favour a latent ODE; strong truncation effects (memory) favour a windowed or recurrent autoregressive model. In both cases validate on roll-outs beyond the training window.

Reduced-Order Modelling

Non-intrusive ROMs: exercises

This is pretty much our starting point from the last exercise: any interpolation-based non-intrusive ROM is pretty bad in the extrapolation task. Only the intrusive ROM can stay decently close to the POD projection. Also, notice that this is our actual target: the POD modes simply cannot predict the FOM cause they are not expressive enough

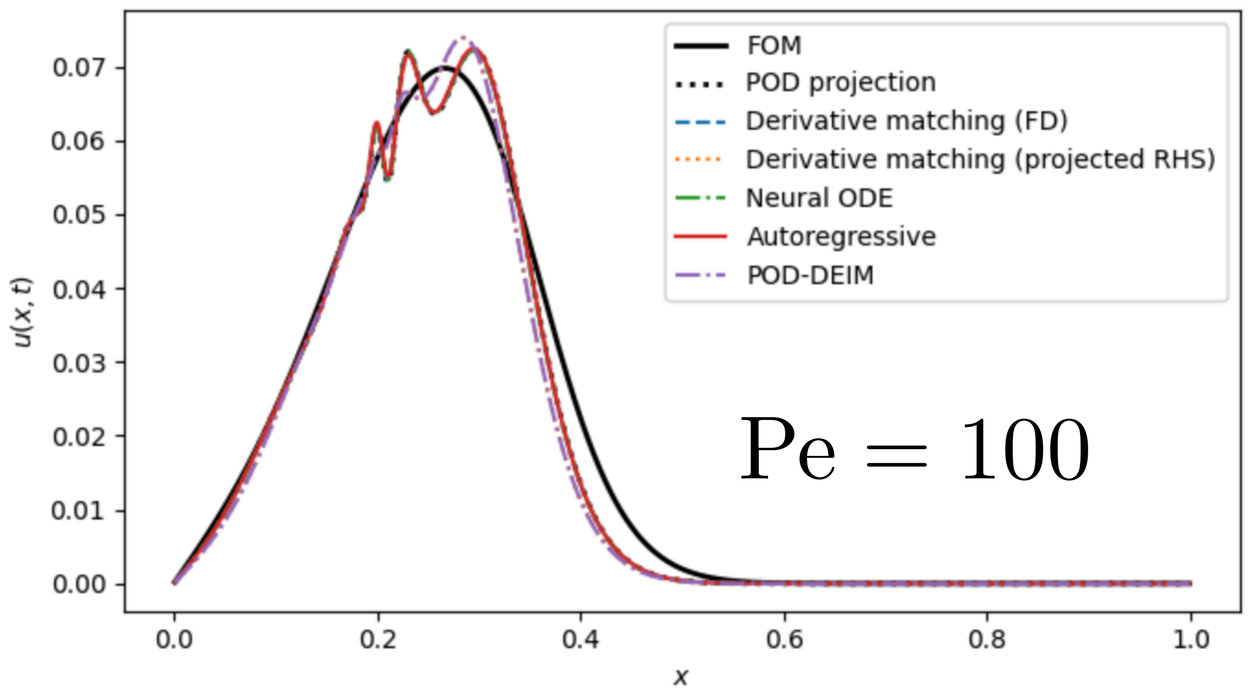


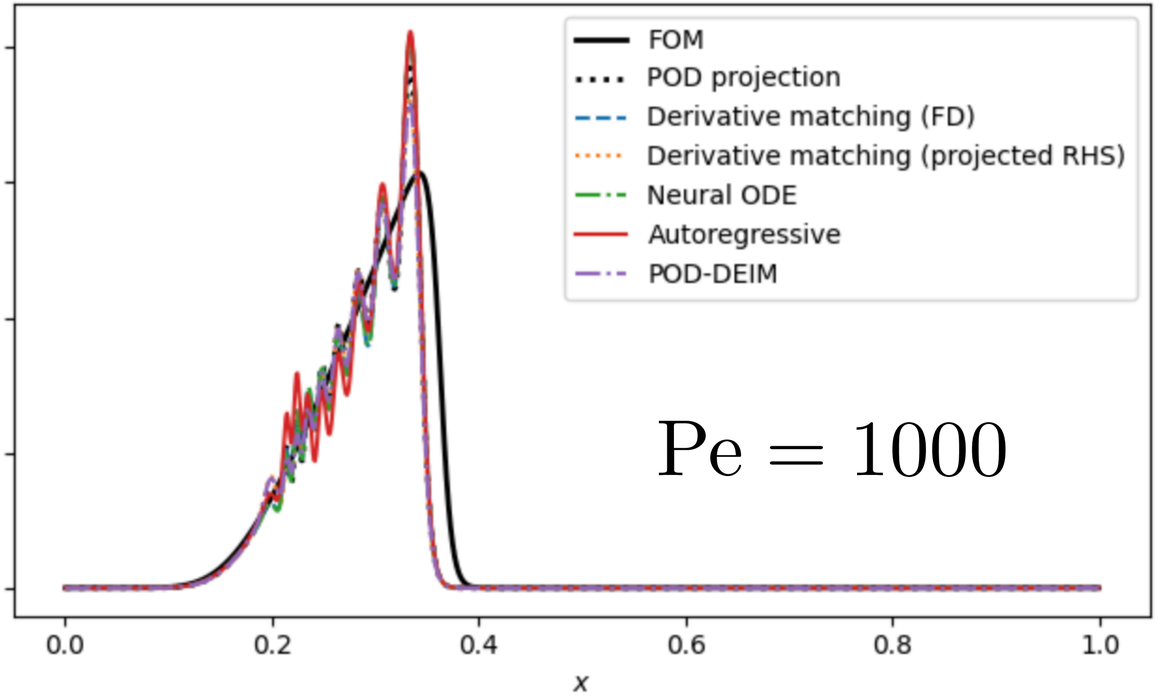
Reduced-Order Modelling

Non-intrusive ROMs: exercises

We compare here the results obtained with the classical non-intrusive ROMs we used up to now and more appropriate ROMs for time-dependent problems. We notice:

- The POD projection is always a little bit off wrt FOM: meaning that POD cannot fully describe these times (projection error). The best we can do is predicting the right modes
- The interpolation-based non intrusive methods are bad in extrapolation (as expected)
- Methods tailor for dynamical systems (NODE, AR etc etc) do a much better job in predicting the POD projection (even better than intrusive ROMs)



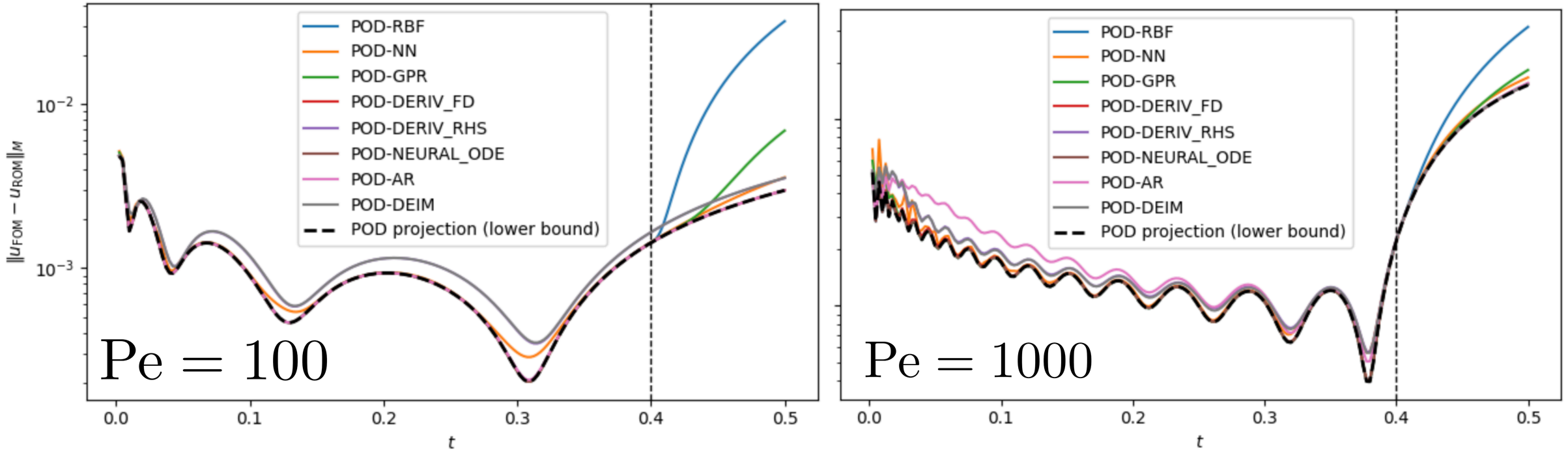


Reduced-Order Modelling

Non-intrusive ROMs: exercises

We compare here the results obtained with the classical non-intrusive ROMs we used up to now and more appropriate ROMs for time-dependent problems. We notice:

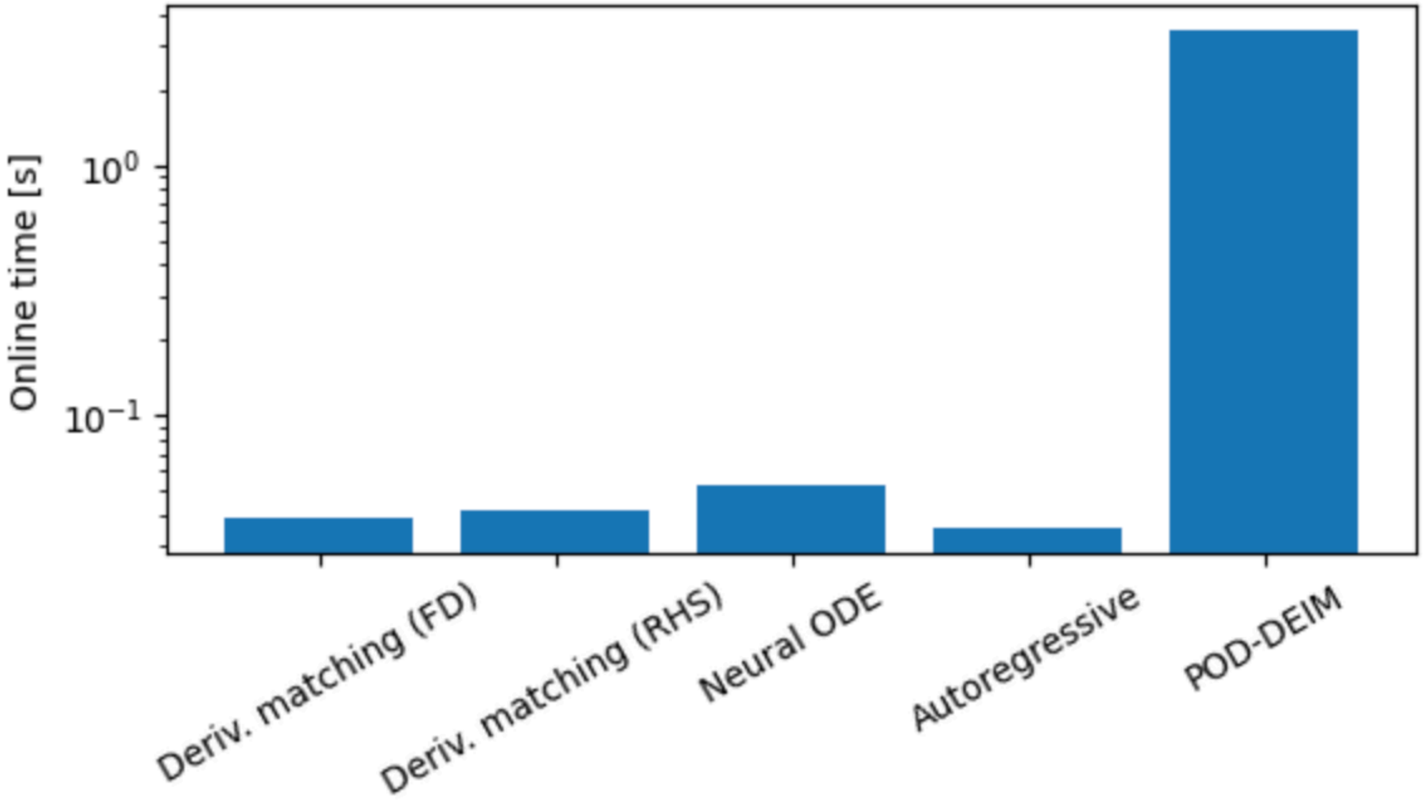
- The POD projection is always a little bit off wrt FOM: meaning that POD cannot fully describe these times (projection error). The best we can do is predicting the right modes
- The interpolation-based non intrusive methods are bad in extrapolation (as expected)
- Methods tailor for dynamical systems (NODE, AR etc etc) do a much better job in predicting the POD projection (even better than intrusive ROMs)



Reduced-Order Modelling

Non-intrusive ROMs: exercises

In fact, the main takeaway message here is that these methods are significantly better than interpolation-based non-intrusive ROMs in terms of extrapolation.
In fact, in terms of accuracy they are comparable to intrusive ROMs. These, even if more physics-based, they still carry the propagation errors of time integration (which can still be further minimised. But the important point is that they do so at a much smaller computational cost:



Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

Kolmogorov n-width

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

Kolmogorov n-width

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

Reading this form the inside out we have:

1. For all functions in a space V_n of dimension n the right-most term gives the minimal distance from the solution u according to a certain measure (projection error like POD)

Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

Kolmogorov n-width

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

Reading this form the inside out we have:

1. For all functions in a space V_n of dimension n the right-most term gives the minimal distance from the solution u according to a certain measure (projection error like POD)
2. Among these distances we take the largest one for all the solutions which belong to our solution manifold (for a fixed subspace)

Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

Kolmogorov n-width

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

Reading this form the inside out we have:

1. For all functions in a space V_n of dimension n the right-most term gives the minimal distance from the solution u according to a certain measure (projection error like POD)
2. Among these distances we take the largest one for all the solutions which belong to our solution manifold (for a fixed subspace)
3. Finally, for all the subspaces of fixed dimension which belong to X, we find the minimum value of such distance.

Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

Kolmogorov n-width

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

Reading this form the inside out we have:

1. For all functions in a space V_n of dimension n the right-most term gives the minimal distance from the solution u according to a certain measure (projection error like POD)
2. Among these distances we take the largest one for all the solutions which belong to our solution manifold (for a fixed subspace)
3. Finally, for all the subspaces of fixed dimension which belong to X, we find the minimum value of such distance.

So, the Kolmogorov n-width gives the errors in the worst case scenario for the best possible linear basis

Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

Kolmogorov n-width

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

For POD we have

$$\min_{\substack{V_n \subset X \\ \dim V_n = n}} \frac{1}{K} \sum_{i=1}^{K} \|u_i - \Pi_{V_n} u_i\|_X^2 = \frac{1}{K} \sum_{i>n} \sigma_i^2, \qquad d_n(\mathcal{S}_K; X) \geq \left(\frac{1}{K} \sum_{i>n} \sigma_i^2 \right)^{1/2}$$

POD minimises a **mean** error and this is related to the disregarded eigenvalues. The Kolmogorov n-width is related to the **worse case scenario**. So, the Kolmogorov n-width will always be larger than the error associated to POD

Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

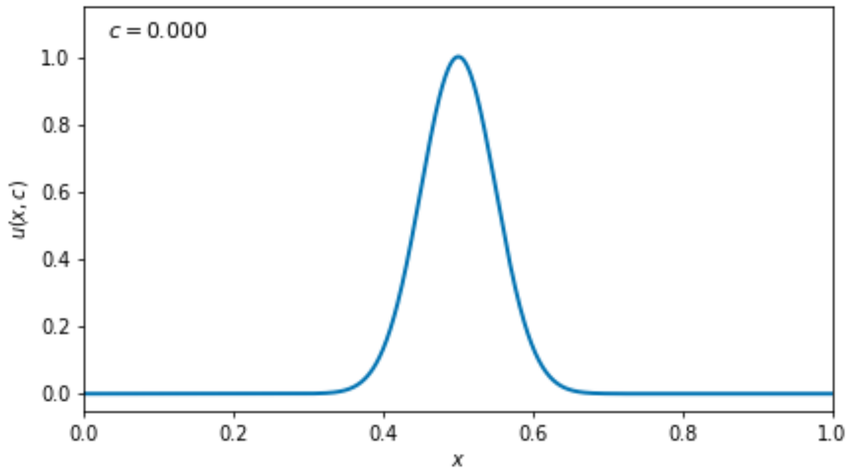
$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$

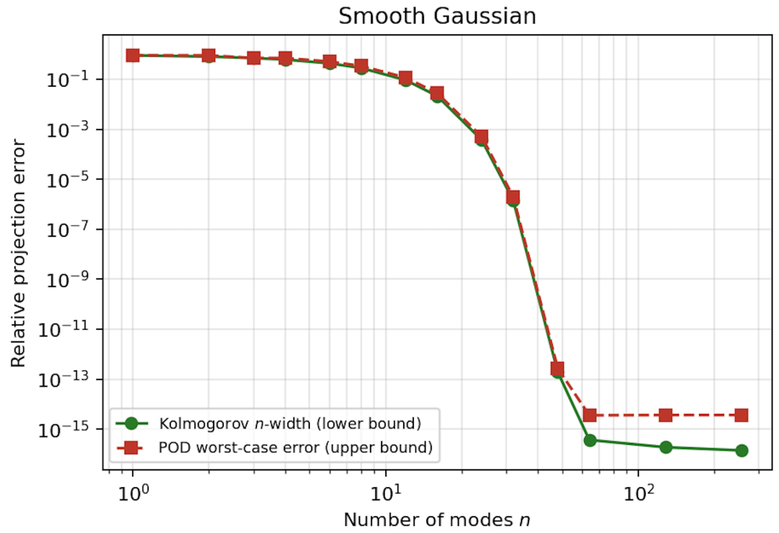
Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$





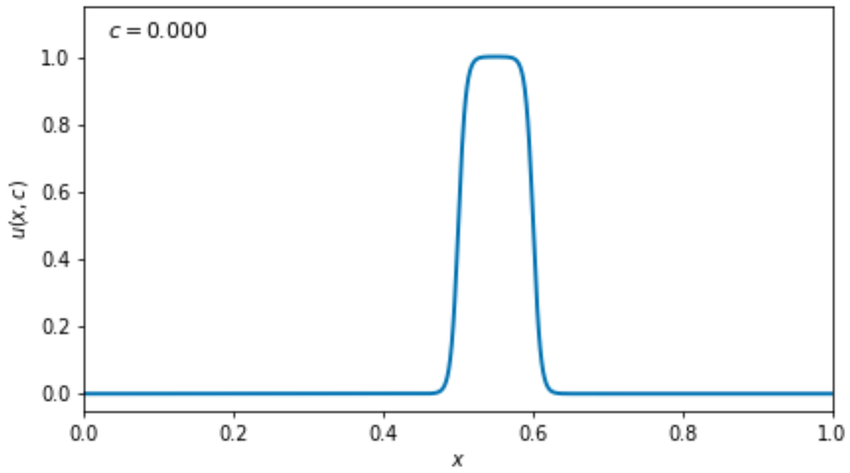
The snapshots at different times are almost orthogonal to each other. Basically, when snapshots are non-overlapping, any linear combination will give a bad approximation. This is intrinsically related to finding a **linear** combination of basis.

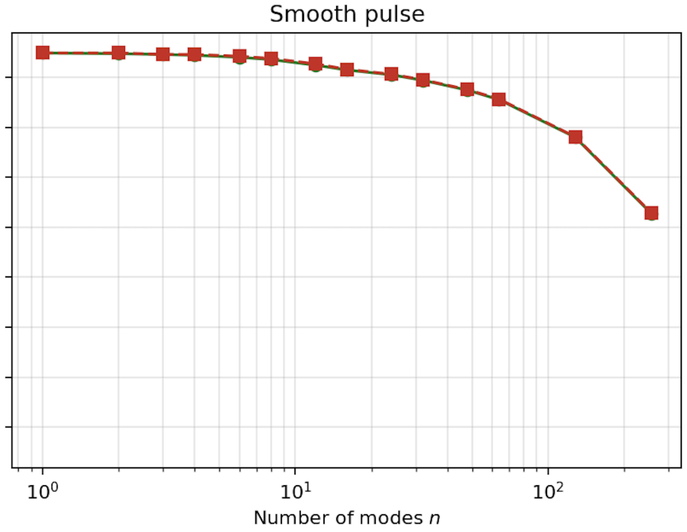
Reduced-Order Modelling

Beyond linear reduction: the Kolmogorov n-width

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

$$d_n(\mathcal{S}; X) = \inf_{\substack{V_n \subset X \\ \dim V_n \leq n}} \sup_{u \in \mathcal{S}} \inf_{v \in V_n} \|u - v\|_X$$





The snapshots at different times are almost orthogonal to each other. Basically, when snapshots are non-overlapping, any linear combination will give a bad approximation. This is intrinsically related to finding a **linear** combination of basis.

Reduced-Order Modelling

Beyond linear reduction:

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

So, for POD we have seen that the reduction is linear: $\mathbf{u}(\mu, t) \approx \mathbf{Za}$

There exist a few extensions of this limitation such as:

1. Quadratic manifold

$$\mathbf{u}(\mu, t) \approx \mathbf{Za} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

2. Local ROMs

3. Kernel POD, shifted POD, registration methods

4. AutoEncoders:

$$\mathbf{u}(\mu, t) \approx NN(\mathbf{z})$$

Reduced-Order Modelling

Beyond linear reduction:

A linear ROM approximates the solution manifold $\mathcal{S} = \{u(\mu, t)\}$ with an n-dimensional **linear** subspace. How good can this ever be?

So, for POD we have seen that the reduction is linear: $\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a}$

There exist a few extensions of this limitation such as:

1. Quadratic manifold

$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds

1. Quadratic manifold

$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds

1. Quadratic manifold

$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

$$\mathbf{Z} \in \mathbb{R}^{\mathcal{N} \times N}, \quad \mathbf{W} \in \mathbb{R}^{\mathcal{N} \times N^2}$$

The matrix $\mathbf{Z}$ comes from POD on the snapshots as always done but how do we compute $\mathbf{W}$?

First we project the snapshots not the POD basis (we still have **linear** encoder)

$$\tilde{\mathbf{u}}(\mu, t)^k \approx \mathbf{Z}\mathbf{a}^k$$

Then we can compute the residual and suppose to model that with a quadratic term:

$$\mathbf{r}^k = \mathbf{u}(\mu, t)^k - \tilde{\mathbf{u}}(\mu, t)^k = \mathbf{u}(\mu, t)^k - \mathbf{Z}\mathbf{a}^k \approx \mathbf{W}(\mathbf{a}^k \otimes \mathbf{a}^k)$$

Now we have the residual, we have the POD coefficients: we only need to find $\mathbf{W}$

Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds

1. Quadratic manifold

$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

$$\mathbf{r}^k = \mathbf{W}(\mathbf{a}^k \otimes \mathbf{a}^k)$$

We can interpret this as a least-squares problem:

$$\min_{\mathbf{W}} \sum_k \|\mathbf{r}^k - \mathbf{W}(\mathbf{a}^k \otimes \mathbf{a}^k)\|^2$$

Note: sometimes some Thikonov regularisation is needed

Defining the matrix of Kronecker products of POD coefficients as

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}^{(1)} \otimes \mathbf{a}^{(1)} & \mathbf{a}^{(2)} \otimes \mathbf{a}^{(2)} & \cdots & \mathbf{a}^{(N_s)} \otimes \mathbf{a}^{(N_s)} \end{bmatrix} \in \mathbb{R}^{N^2 \times N_s}.$$

And the matrix containing the residual as

$$\mathbf{R} = \begin{bmatrix} \mathbf{r}^{(1)} & \mathbf{r}^{(2)} & \cdots & \mathbf{r}^{(N_s)} \end{bmatrix} \in \mathbb{R}^{N \times N_s}.$$

We get:

$$\mathbf{W} = \mathbf{R}\mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1}$$

Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds

1. Quadratic manifold

$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

Now the pipeline is very similar with respect to standard linear ROMs:

1. Collect snapshots
2. Apply POD to get the matrix $\mathbf{Z}$
3. Solve the regression problem to get the matrix $\mathbf{W}$

Then similarly to standard ROMs you an decided how to predict the POD coefficients. The intrusive ROM is a bit more complex due to the presence of the quadratic form while non-intrusive is extremely straightforward:

1. You collect a series of realisations of the mapping $\mathcal{T} = \{(\boldsymbol{\mu}_i,\ \mathbf{a}_i)\}_{i=1}^{n}, \qquad \boldsymbol{\mu} \longmapsto \mathbf{a}(\boldsymbol{\mu})$
2. You train your interpolation/regression model and you can obtain new POD coefficients for new parameters and reconstruct the solution

$$\boldsymbol{\mu}^* \longmapsto \mathbf{a}(\boldsymbol{\mu}^*) \longrightarrow \mathbf{u}(\mu^*, t) \approx \mathbf{Z}\mathbf{a}^* + \mathbf{W}(\mathbf{a}^* \otimes \mathbf{a}^*)$$

Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds

1. Quadratic manifold

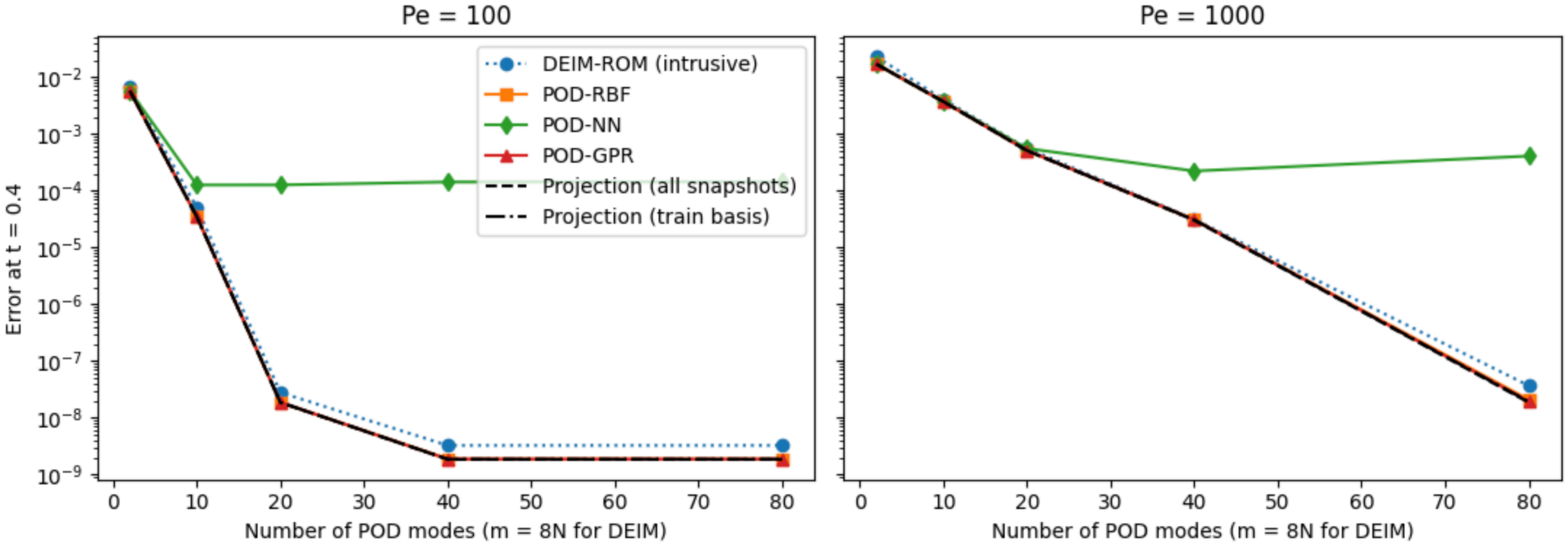
$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$

So, in practice, as a summary, using quadratic ROMs changes only the **decoder** meaning that the reconstruction is based on the above equation, while the **encoder** is still **linear** we are still using only the POD modes. Or, if you prefer, we are projecting the solution in the POD space but then we **correct** the linear prediction with a quadratic correction which is based on the residual

Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds (results)

These are the results we got from non-intrusive ROMs in the interpolation regime in time, as simple as that. We noticed last time that we struggles a little in get good results using the NN but every other method is pretty good and it's even better then DEIM-ROM.

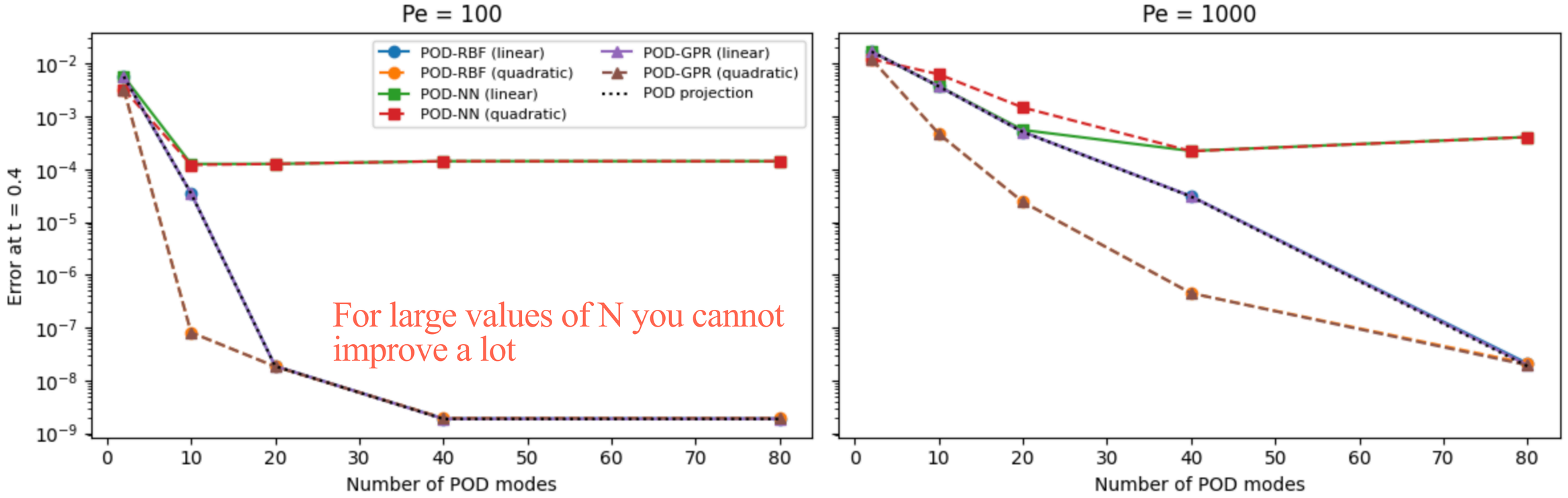


Reduced-Order Modelling

Beyond linear reduction: quadratic manifolds (results)

If we implement the quadratic reconstruction we are not changing ANYTHING in the prediction of the coefficients: it only changes the final reconstruction as

$$\mathbf{u}(\mu, t) \approx \mathbf{Z}\mathbf{a} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$$



Reduced-Order Modelling

Beyond linear reduction: other non-linear reductions

Local ROMs:

Cluster the snapshots, build one POD basis per cluster, switch basis online according to the current state

Transport aware:

Remove the transport first, shifted POD, registration and mapping methods, then compress what is left

Common idea: a better (nonlinear) reconstruction, or a coordinate change, so that the manifold becomes easier to describe with few variables

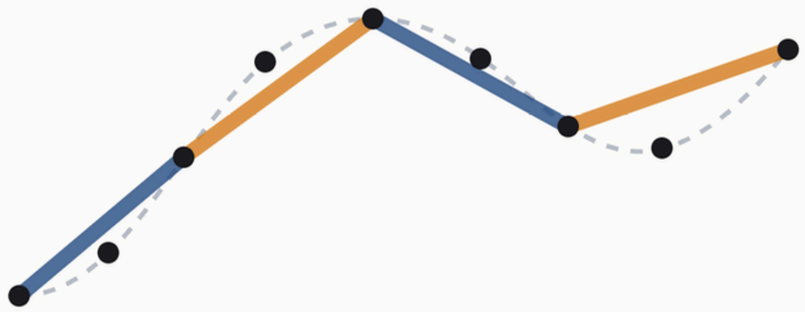
Reduced-Order Modelling

Beyond linear reduction: other non-linear reductions

Local ROMs:

Cluster the snapshots, build one POD basis per cluster, switch basis online according to the current state

1. **Cluster the snapshots**
   k-means on the snapshots or on the parameters: each cluster is one regime.
2. **One POD basis per cluster**
   Offline: a small basis V for every cluster, since each regime is simpler.
3. **Switch online**
   Find the cluster of the current state or parameter, then use $\mathbf{u} \approx \mathbf{Z}\mathbf{a}$ with that basis.



Snapshots on a curved manifold, covered by flat local patches

Gains. Distinct regimes need few modes each.

Costs. Switching can be discontinuous; more bases and snapshots.

Reduced-Order Modelling

Beyond linear reduction: other non-linear reductions

Transport aware:

Remove the transport first, shifted POD, registration and mapping methods, then compress what is left

Reduced-Order Modelling

Beyond linear reduction: other non-linear reductions

Transport aware:

Remove the transport first, shifted POD, registration and mapping methods, then compress what is left

Reduced-Order Modelling

Beyond linear reduction: other non-linear reductions

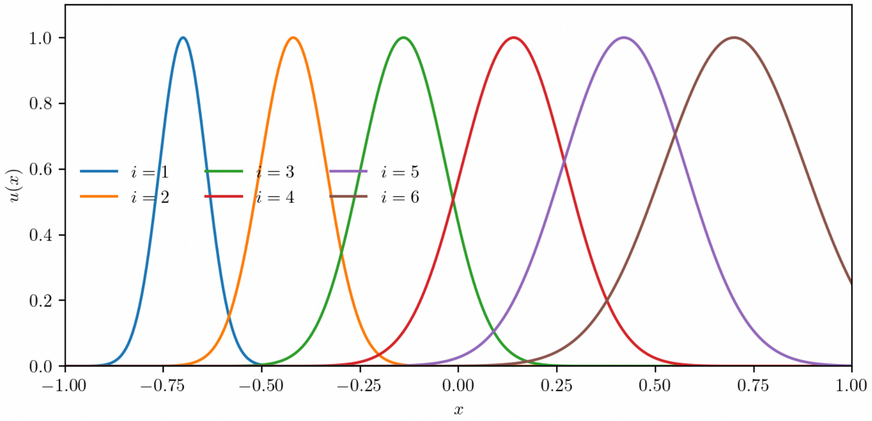
Transport aware:

Remove the transport first, shifted POD, registration and mapping methods, then compress what is left

The idea of transport aware approach like shifted POD or registration methods are based on finding a non-linear map to transform the snapshots so that POD works well, apply POD in this transformed space and then apply the inverse mapping to recover the solution.

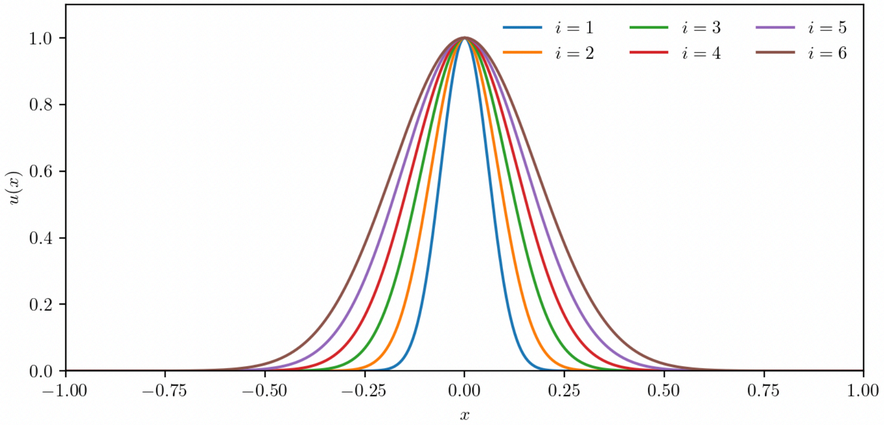
They work very well but for quite limited case where it is “easy” to define such a mapping

POD will work quite badly applied to this dataset:



Can I find a map so that POD works well?

Like in this setting



Reduced-Order Modelling

Beyond linear reduction: other non-linear reductions

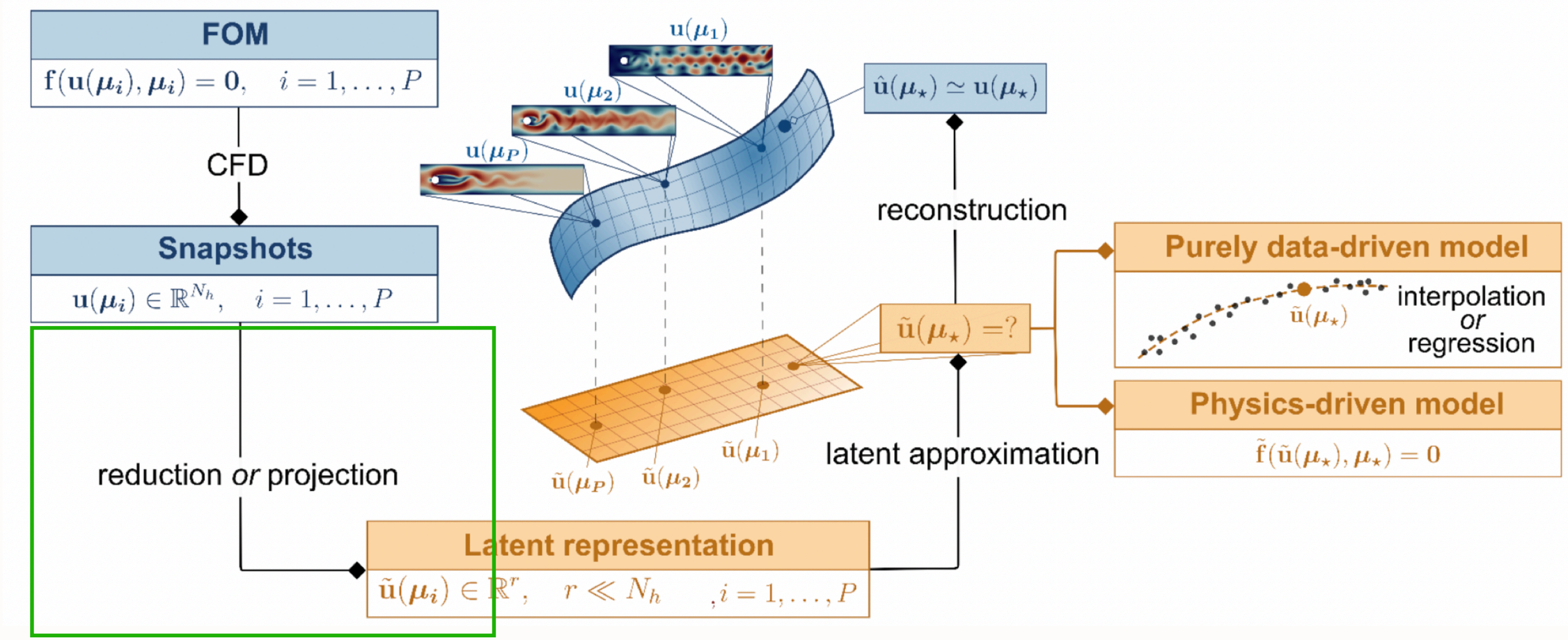
Comparison of the non-linear ROMs

Method	What changes	Strength	Limitation
Quadratic manifold	Decoder gains a quadratic term	Closed form, cheap to train	Only quadratic curvature
Local ROMs	One basis per region	Handles distinct regimes	Switching, more snapshots
Transport-aware	Coordinates	Advection-dominated problems	Transport must be identifiable

Next. Autoencoders learn the nonlinear map itself, with no assumption on its form.

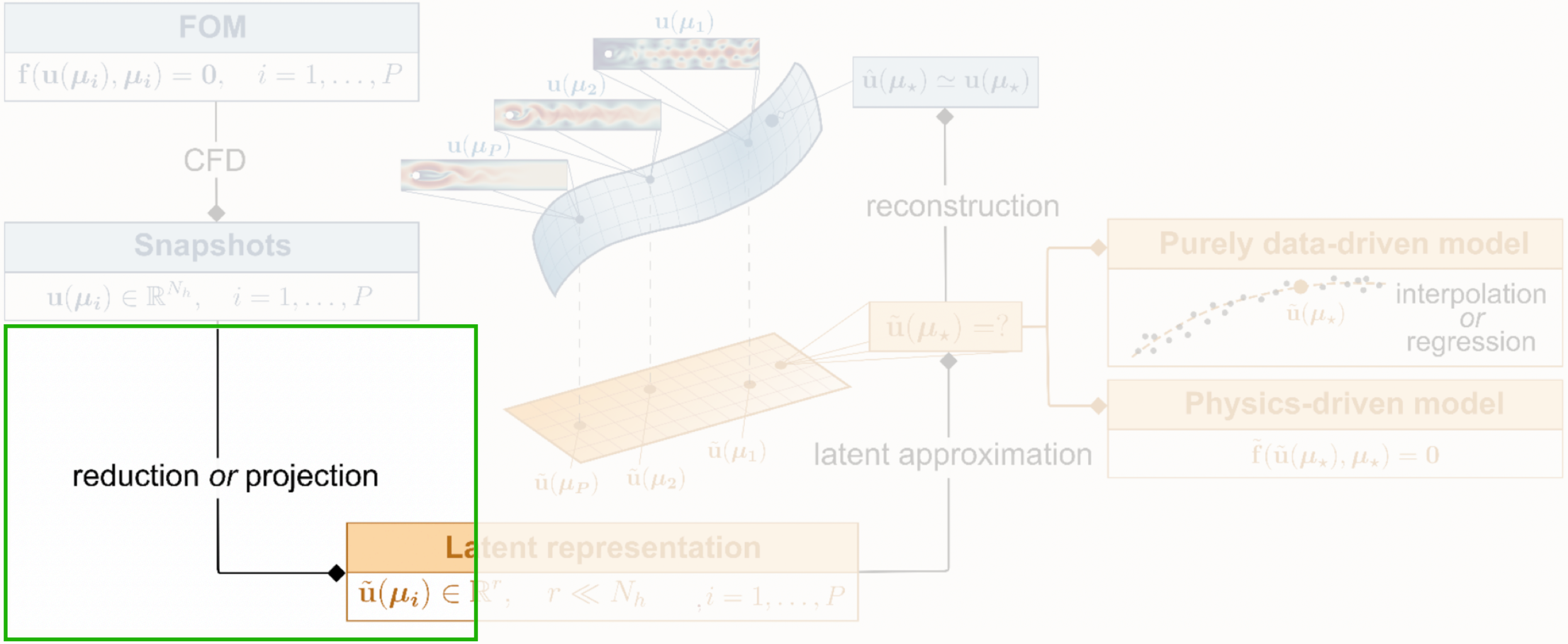
Reduced-Order Modelling

Beyond linear reduction: AutoEncoders



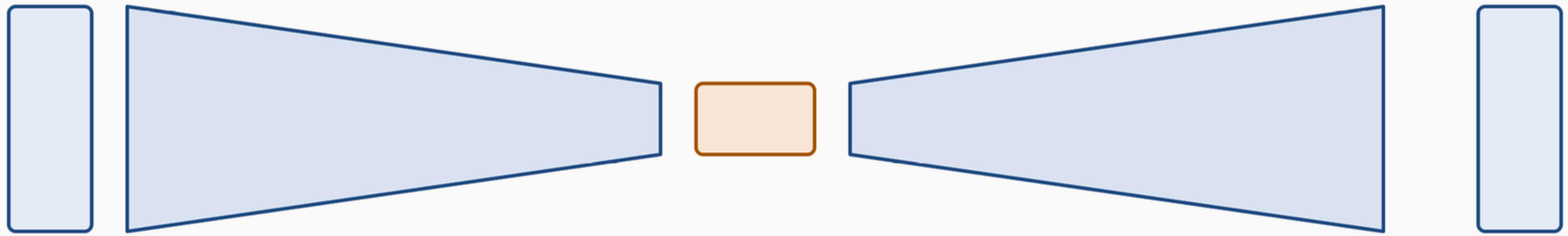
Reduced-Order Modelling

Beyond linear reduction: AutoEncoders



Reduced-Order Modelling

Beyond linear reduction: AutoEncoders



Snapshot $\mathbf{u} \in \mathbb{R}^{\mathcal{N}}$ Encoder E Latent $\mathbf{z} \in \mathbb{R}^{N}$ Decoder D Reconstruction

TRAINING

Minimise the mean of $\|\mathbf{u} - D(E(\mathbf{u}))\|^2$ over the snapshots.

REDUCTION

The bottleneck $\mathbf{z} \in \mathbb{R}^{N}$, with $N << \mathcal{N}$, is the ROM state.

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

Linear versus non-linear latent spaces

Linear autoencoder = POD

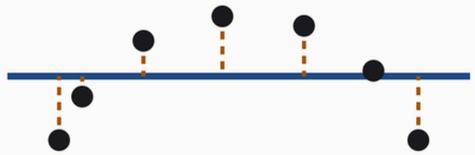
With linear maps and the squared error, the optimum spans the POD subspace. The basis need not be orthogonal or ordered.

Non-linear activations

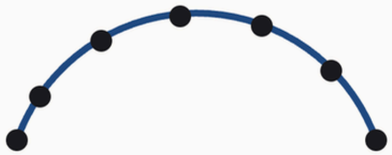
The range of the decoder becomes a curved r-dimensional manifold.

A hierarchy of decoders

Linear $\mathbf{Za}$, quadratic $\mathbf{Za} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$, neural network $D(\mathbf{z})$.



Linear: one mode leaves a large error



Non-linear: one latent coordinate is enough

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

Architectures and training

FEW DEGREES OF FREEDOM

Fully connected

A plain MLP on the full state or on POD coefficients.

STRUCTURED GRIDS

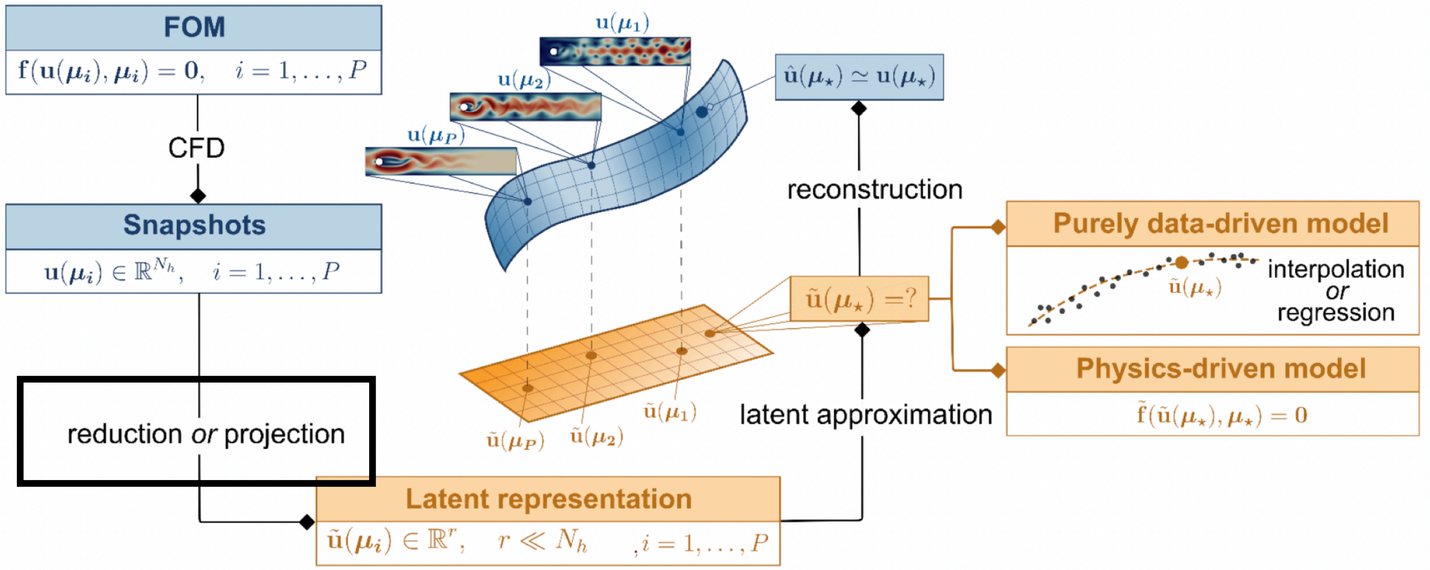
Convolutional

Shared filters exploit spatial structure with few parameters.

UNSTRUCTURED MESHES

Graph-based

Graph convolutions act directly on the mesh connectivity.



Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

Architectures and training

FEW DEGREES OF FREEDOM

Fully connected

A plain MLP on the full state or on POD coefficients.

STRUCTURED GRIDS

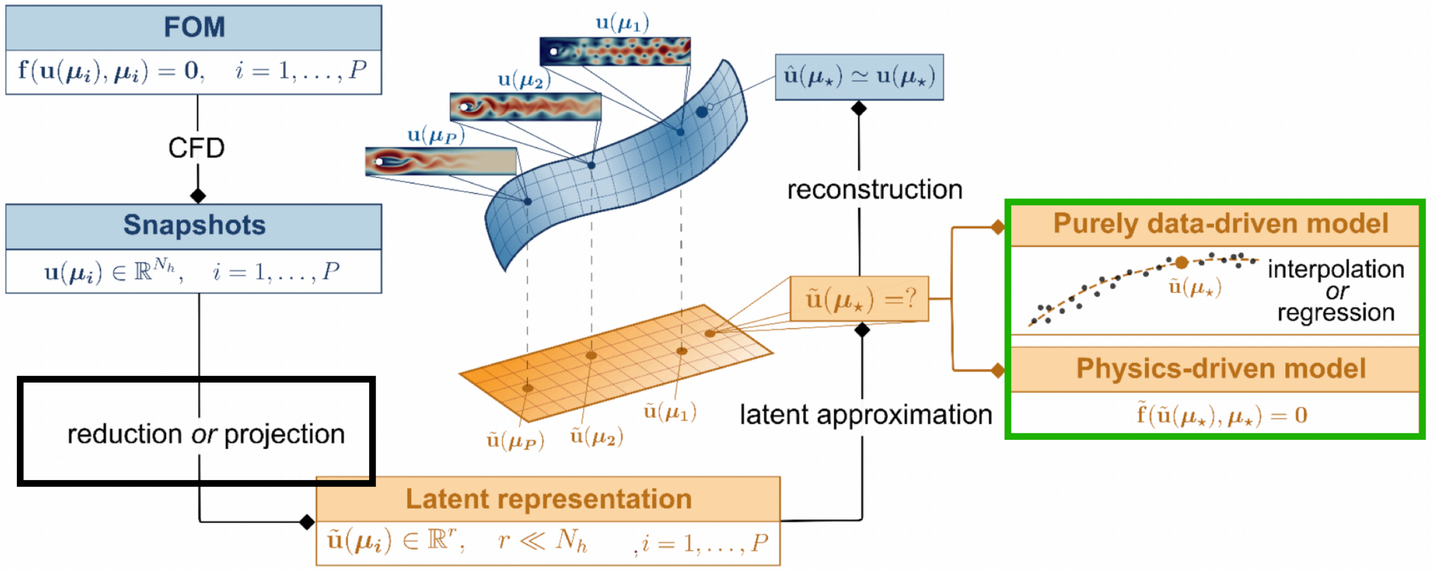
Convolutional

Shared filters exploit spatial structure with few parameters.

UNSTRUCTURED MESHES

Graph-based

Graph convolutions act directly on the mesh connectivity.



Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

Non-intrusive ROMs with autoencoders

Offline

Training snapshots $\mathbf{u}(\mu)$ → Train the autoencoder → Encode: $\mathbf{z} = E(\mathbf{u})$ → Regress (GPR, RBF, NN) $\mu \to \mathbf{z}$

Online

New parameter μ^* → Predict the latent state $\mathbf{z}(\mu^*)$ → Decode: $\tilde{\mathbf{u}} = D(\mathbf{z}(\mu^*))$

Same pipeline as before. Only the reduction and reconstruction maps change: E and D replace projection and lift. The latent coordinates must vary smoothly with μ for the regression to work.

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

Autoencoders in intrusive ROMs

FULL MODEL

$$\frac{\partial \mathbf{u}}{\partial t} = f(\mathbf{u})$$

ANSATZ

$$\mathbf{u} \approx D(\mathbf{z})$$

CHAIN RULE

$$dD\frac{\partial \mathbf{z}}{\partial t} = f(D(\mathbf{z})) \qquad dD = \frac{\partial D}{\partial \mathbf{z}}$$

REDUCED SYSTEM

$$\frac{\partial \mathbf{z}}{\partial t} = (dEdD)^{-1}dEf(D(\mathbf{z}))$$

A few comments on this:

- For the POD, the product of the Jacobians is simply $Z^T Z$ and if you build the POD appropriately this is simply the identity matrix.
- For Autoencoders that product tends to be close to the identity but it not always is
- The last term on the rhs involves operations scaling with the number of DoF of the FOM: so you need some kind of hype reduction

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

Strengths and costs of autoencoders

Strengths

- Latent dimension close to the intrinsic dimension
- Breaks the linear n-width barrier: fronts, advection
- Cheap online: one decoder evaluation
- Plugs into the usual non-intrusive pipeline

Costs

- Non-convex training, no optimality guarantee
- Needs many snapshots; poor extrapolation
- Latent coordinates are not ordered by energy
- Intrusive use needs decoder Jacobians

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders

From POD to autoencoders

Model	Decoder	How it is built	Guarantee
POD	Linear: $\mathbf{Za}$	SVD	Optimal among linear maps
Quadratic manifold	$\mathbf{Za} + \mathbf{W}(\mathbf{a} \otimes \mathbf{a})$	Linear least squares	Convex fit, cheap
Autoencoder	Neural network $D(\mathbf{z})$	Non-convex training	None: validate on unseen data

Rule of thumb. Add non-linearity only as far as the solution manifold requires.

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders (exercises)

OFFLINE: learn from snapshots

```
Input: snapshots u_i at times t_i, latent dim N
1. split: hold out t*, train on rest
   S_tr, t_tr <- snapshots without t*
2. encode to N latent coordinates
   POD: Z <- eig(S^T M S), Z^T M Z = I
        q_i <- Z^T M u_i
   AE : train E, D on y = L^T u
        min sum ||D(E(y_i)) - y_i||^2
        q_i <- E(u_i)
3. regress time -> latent coordinates
   q^(t) <- fit{RBF | NN | GPR}(t_tr, q_tr)
```

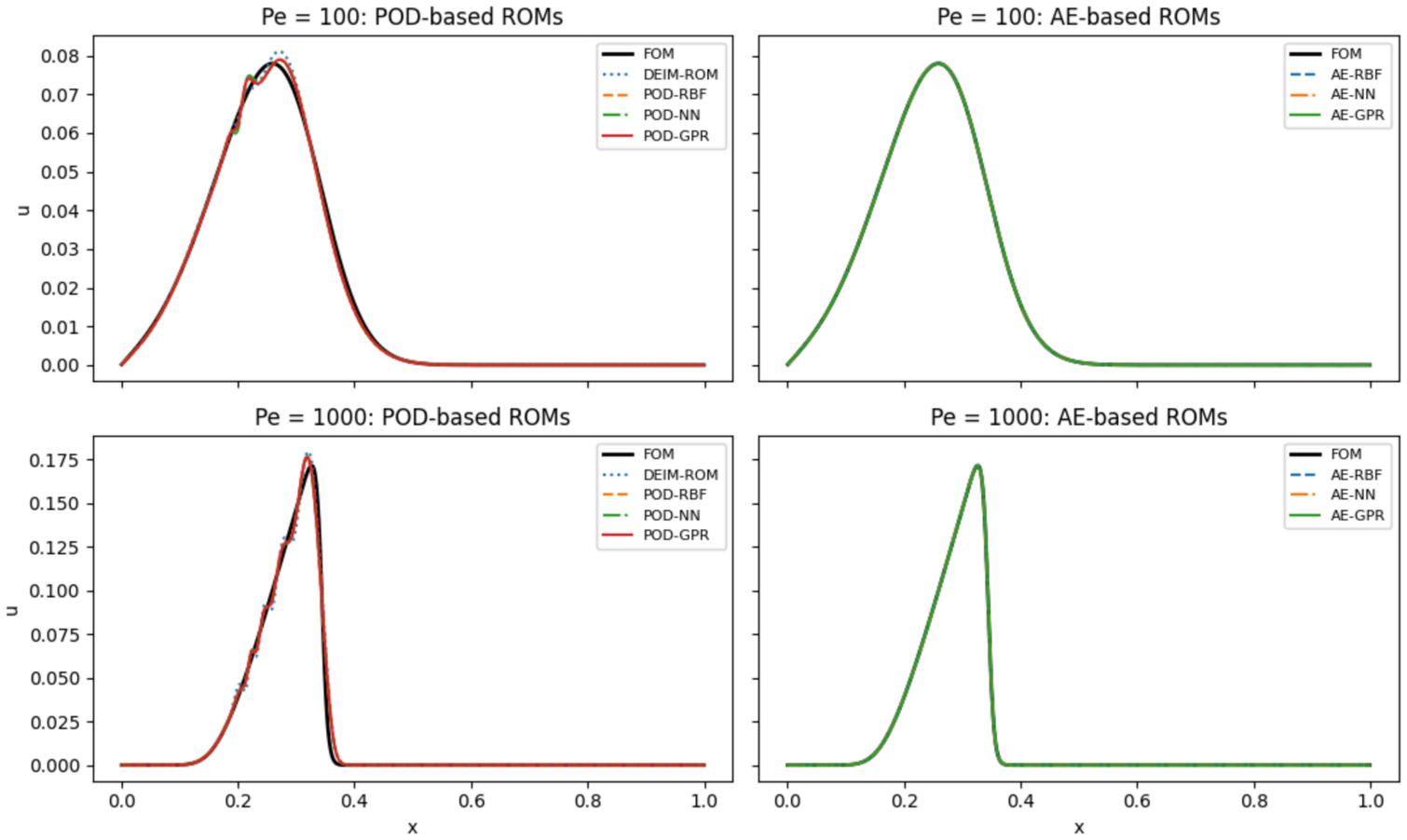
ONLINE: predict at new t*

```
Input: new time t*
1. predict latent state
   q^ <- q^(t*)
2. decode to full state
   POD: u^ <- Z q^
   AE : u^ <- D(q^)
3. return u^
   err = ||u - u^||_M
```

Reduced-Order Modelling

Beyond linear reduction: AutoEncoders (exercises)

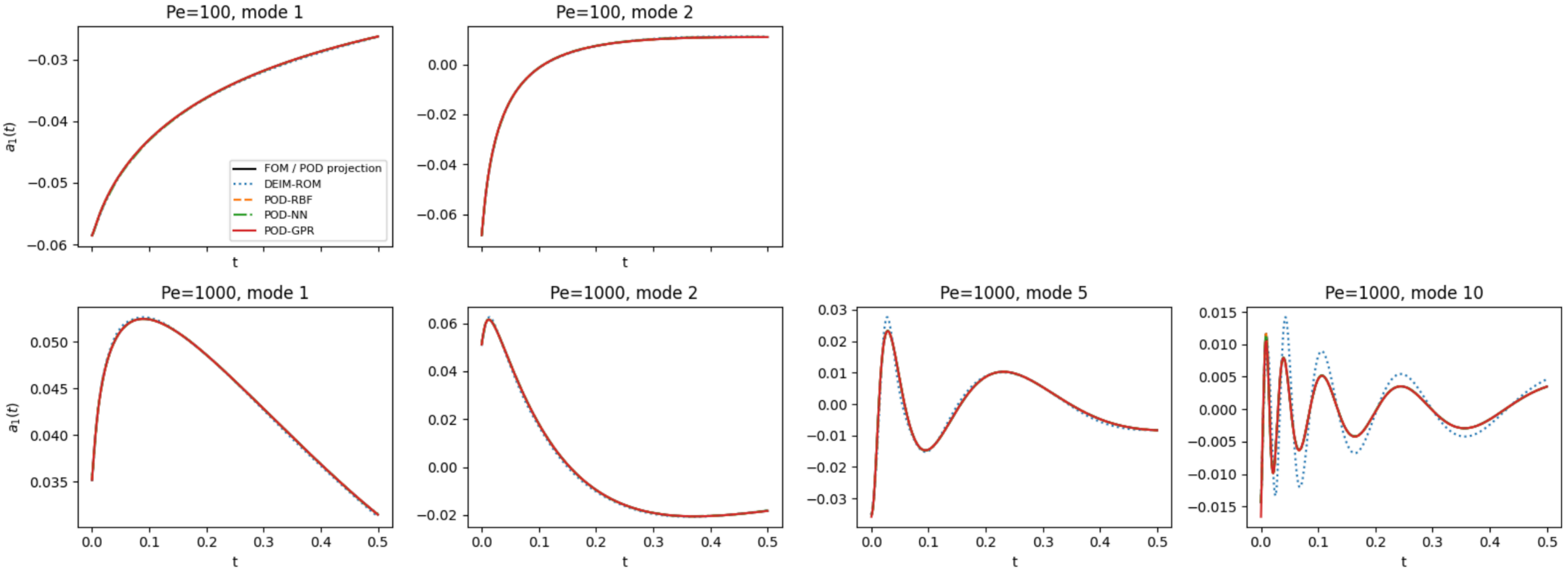
In order to make a fair comparison I kept the size of the low dimensional space fixed and built ROMs based on POD and AEs for our little test case.



Reduced-Order Modelling

Beyond linear reduction: AutoEncoders (exercises)

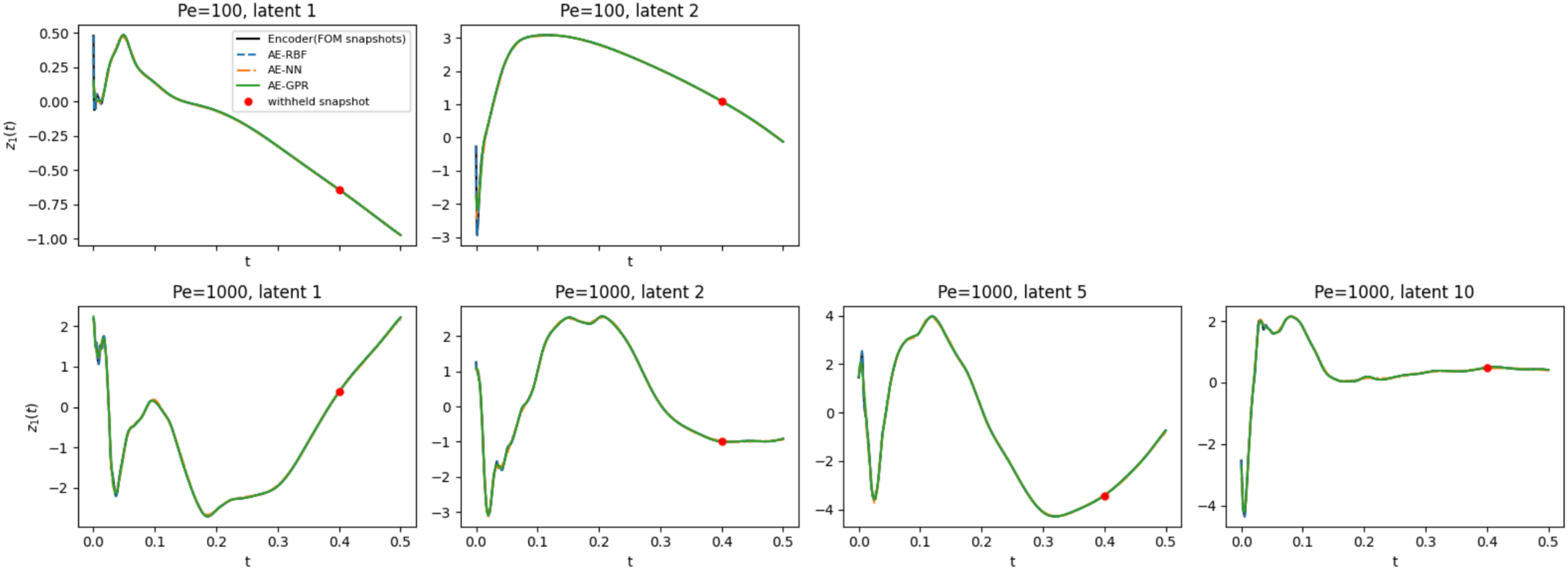
Comparison of the POD modes



Reduced-Order Modelling

Beyond linear reduction: AutoEncoders (exercises)

Comparison of the POD modes



Reduced-Order Modelling

Beyond linear reduction: AutoEncoders (exercises)

Comparison of the POD modes

